

Handheld Device Programming I

Submission Guidelines

There are three (03) components to this assessment. **All components must be compiled and submitted as one ZIP folder.**

Task 01: Chat Application Development using React Native and Java Servlet Backend

- **Objective:** To develop a fully functional chat application demonstrating real-time communication using React Native for the frontend and Java Servlet for the backend.
- **Submission Format: Zip File (source code and related files)**

Task 02: Project Documentation for the Chat Application

- **Objective:** To prepare a detailed documentation report covering
 1. System design
 2. Architecture
 3. Diagrams
 4. Implementation details of the chat application.
- **Submission Format: PDF Document**

Task 03: Tour Video Demonstration

- **Objective:** To create a video presentation explaining the chat application's features, backend workflow, and overall project functionality.

IMPORTANT GUIDELINES FOR CANDIDATES

- Maximum video duration: 10 minutes
- The video must be presented in English.
- Use the Zoom platform to record the video (Students must turn on their camera, ensuring a clear view of their face during the demonstration).
- Student Introduction: State your name and Student Portal registered NIC number. The video must include a thumbnail image containing the student's name, NIC, photo, subject, and assignment details. (Duration: 30 seconds)
- File Naming Conventions:
 - Rename the PDF file as: HHDP_I_ASDOC-Your NIC-Your Name
 - Rename the video file as: HHDP_I_ASVID-Your NIC-Your Name



HANDHELD DEVICE PROGRAMMING I
(REACT NATIVE)
CHAT APPLICATION
PROJECT DOCUMENTATION

STUDENT NAME :
NIC :
PROJECT NAME :

Acknowledgement

*This section is to **express gratitude** to everyone who helped in the successful completion of the project. It may include thanks to faculty members, project guides, colleagues, friends, or family who provided technical guidance, resources, or moral support.*

Table of Contents

Acknowledgement 1

Table of Figures 3

Project Overview 3

Objectives 3

Technology Stack..... 3

System Architecture 3

Features 3

User Interface (UI) Design..... 4

Diagrams 4

Backend Implementation 4

Frontend Implementation..... 4

Database Design..... 5

Security Measures..... 5

Testing..... 5

Deployment..... 5

Future Enhancements 5

Conclusion 5

Table of Figures

This section provides a list of all diagrams, charts, and figures used in the documentation along with their titles and page numbers. It helps readers quickly locate visual content such as system diagrams and database models.

Project Overview

The overview provides a summary of the project, including the purpose and scope. For example, the chat application allows users to send and receive real-time messages, similar to WhatsApp or Messenger, but implemented as a learning project. It also introduces the platforms (Android/iOS) the app supports.

Objectives

The objectives explain what the project aims to achieve. Example objectives:

- *To build a real-time communication system.*
- *To ensure simple and user-friendly design.*
- *To use secure authentication and message handling.*
- *To practice using React Native for frontend and Java + Hibernate + WebSocket for backend.*

Technology Stack

This part explains the tools and technologies used:

- *Frontend: React Native with JavaScript/TypeScript for cross-platform mobile app development.*
- *UI Styling: NativeWind or hardcoded styles for customizing screens.*
- *Backend: Java Servlet for handling API requests and Hibernate for database ORM (mapping objects to relational tables).*
- *Communication: WebSocket for real-time data transfer and Fetch API for REST calls.*
- *Database: MySQL/PostgreSQL for storing user and chat data.*

System Architecture

The system architecture describes the overall structure of the application:

- *Client Layer (Mobile App): Built using React Native, where users interact.*
- *Server Layer (Backend): Java Servlets + Hibernate for processing requests and storing data.*
- *Communication Layer: WebSocket ensures instant data transfer between client and server.*
- *Database Layer: Stores users, messages, and chat sessions.*

Features

Lists the functionalities available in the chat app. Examples:

- *User login and signup.*
- *Real-time chat using WebSocket.*
- *Chat list showing recent conversations.*
- *Individual chat window for private messaging.*
- *Animations for smooth UI transitions.*
- *Alerts for new messages, errors, or logout.*

User Interface (UI) Design

Explain the design of app screens:

- *Login/Signup Screen – user authentication.*
- *Chat List Screen – list of all conversations.*
- *Individual Chat Screen – conversation between two users.*
- *Profile/Settings Screen – update user info.*
- *Also describes the styling approach (NativeWind or inline styles) to ensure consistent layout.*

Diagrams

This section contains visual diagrams for better understanding:

- *ER Diagram (Entity-Relationship): Shows database tables (User, Chat, Message) and relationships.*
- *Use Case Diagram: Shows what actions users can perform (login, send message, receive message).*
- *Class Diagram: Shows backend classes like User, Chat, Message, and WebSocketHandler, and their relationships.*
- *Sequence Diagram (Optional): Shows step-by-step flow (User sends message → Server receives → Forward to recipient).*

Backend Implementation

Explain how the server-side logic works:

- *User login/signup handled by Java Servlet APIs.*
- *Hibernate is used to map objects (User, Chat) into relational database tables.*
- *WebSocket is used for sending messages instantly.*
- *REST APIs used for fetching chat history.*

Frontend Implementation

Covers how the mobile app frontend is built:

- *React Native components for UI (chat bubbles, lists, buttons).*
- *State management to update chat lists and messages dynamically.*
- *WebSocket integration to connect with backend for live chat.*
- *Animations & Alerts for smooth user experience.*

Database Design

Details about database structure:

- *User Table: Stores id, username, password, contact number.*
- *Chat Table: Stores chatId, participants, and timestamp.*
- *Message Table: Stores messageId, chatId, senderId, text, timestamp, status.*

This section usually includes an ER Diagram to visualize relationships.

Security Measures

Explains how the system is kept secure:

- *Authentication with tokens/session handling.*
- *Securing WebSocket connections.*
- *Preventing unauthorized access to chat data.*

Testing

Covers the testing process:

- *Unit Testing: Individual components (e.g., message send function).*
- *Integration Testing: Ensures frontend and backend communicate correctly.*
- *UI Testing: Check that screens render properly on different devices.*
- *Load Testing (Optional): Test how many users the system can handle at once.*

Deployment

Explain how the system is deployed:

- *Backend: Deployed on GlassFish/Tomcat server with database hosting.*
- *Frontend: React Native app built as APK (for Android) or IPA (for iOS).*
- *Distribution: App can be distributed for internal testing or published to Play Store/App Store.*

Future Enhancements

Lists the planned improvements for the app:

- *Push notifications for new messages.*
- *Voice and video calling features.*
- *Group chats and media sharing (images, files).*
- *Dark mode for better UI.*

Conclusion

Summarizes the overall achievement of the project, what was learned during development, and its practical use. This section also reflects on how the app can be extended in the future.